

Week 2 Exercises

Nick Hegge

July 11, 2024

Please complete all exercises below. You may use stringr, lubridate, or the forcats library.

Place this at the top of your script: library(stringr) library(lubridate) library(forcats)

Exercise 1

Read the sales_pipe.txt file into an R data frame as sales.

```
library(stringr)
library(lubridate)
```

```
##
## Attaching package: 'lubridate'

## The following objects are masked from 'package:base':
##
##      date, intersect, setdiff, union
```

```
library(forcats)
```

```
data <- read.csv("C:/Users/Nhegg/OneDrive/Documents/Database/Merrimack College - Nick Masters/DSE5002/Merrimack College Sales Data/sales_pipe.txt",
                 stringsAsFactors = FALSE, fileEncoding = "latin1")
```

Exercise 2

You can extract a vector of columns names from a data frame using the colnames() function. Notice the first column has some odd characters. Change the column name for the FIRST column in the sales date frame to Row.ID.

Note: You will need to assign the first element of colnames to a single character.

```
colnames(data)[1] <- "Row.ID"
```

Exercise 3

Convert both Ship.Date and Order.Date to date vectors within the sales data frame. What is the number of days between the most recent order and the oldest order? How many years is that? How many weeks?

Note: Use lubridate

```
# Convert Ship.Date and Order.Date to date vectors
data$Ship.Date <- as_date(data$Ship.Date, format = "%m/%d/%Y")
data$Order.Date <- as_date(data$Order.Date, format = "%m/%d/%Y")

# Calculate number of days between the most recent order and the oldest order
days_diff <- as.numeric(difftime(max(data$Order.Date), min(data$Order.Date), units = "days"))

# Calculate number of years and weeks
years_diff <- days_diff / 365.25
weeks_diff <- days_diff / 7

# Print
cat("Number of days between the most recent order and the oldest order:", days_diff, "days\n")
```

```
## Number of days between the most recent order and the oldest order: 1457 days
```

```
cat("Number of years:", years_diff, "years\n")
```

```
## Number of years: 3.989049 years
```

```
cat("Number of weeks:", weeks_diff, "weeks\n")
```

```
## Number of weeks: 208.1429 weeks
```

Exercise 4

What is the average number of days it takes to ship an order?

```
# Average
average_days_to_ship <- mean(data$Ship.Date - data$Order.Date, na.rm = TRUE)

# Print
cat("Average number of days it takes to ship an order:", round(average_days_to_ship, 2), "days\n")
```

```
## Average number of days it takes to ship an order: 3.96 days
```

Exercise 5

How many customers have the first name Bill? You will need to split the customer name into first and last name segments and then use a regular expression to match the first name bill. Use the length() function to determine the number of customers with the first name Bill in the sales data.

```

# Split customer name into first and last name
data$Customer.FirstName <- word(data$Customer.Name, 1)

# Find 'Bill'
bill_customers <- str_detect(data$Customer.FirstName, regex("^Bill$", ignore_case = TRUE))

# Count customers with the first name Bill
number_of_bills <- length(which(bill_customers))

# Print
cat("Number of customers with the first name Bill:", number_of_bills, "\n")

```

```
## Number of customers with the first name Bill: 72
```

Exercise 6

How many mentions of the word 'table' are there in the Product.Name column? **Note you can do this in one line of code**

```

# Count 'table' in the Product.Name column
table_mentions <- sum(str_count(data$Product.Name, regex("table", ignore_case = TRUE)))

# Print
cat("Number of mentions of the word 'table' in the Product.Name column:", table_mentions, "\n")

```

```
## Number of mentions of the word 'table' in the Product.Name column: 604
```

Exercise 7

Create a table of counts for each state in the sales data. The counts table should be ordered alphabetically from A to Z.

```

# table of counts for each state
state_counts <- table(data$State)

# Convert to order alphabetically
state_counts_df <- as.data.frame(state_counts)
state_counts_df <- state_counts_df[order(state_counts_df$Var1), ]

# Print
print(state_counts_df)

```

```
##           Var1 Freq
## 1      Alabama   61
## 2      Arizona  224
## 3    Arkansas   60
## 4   California 2001
## 5      Colorado  182
## 6   Connecticut   82
```

## 7	Delaware	96
## 8	District of Columbia	10
## 9	Florida	383
## 10	Georgia	184
## 11	Idaho	21
## 12	Illinois	492
## 13	Indiana	149
## 14	Iowa	30
## 15	Kansas	24
## 16	Kentucky	139
## 17	Louisiana	42
## 18	Maine	8
## 19	Maryland	105
## 20	Massachusetts	135
## 21	Michigan	255
## 22	Minnesota	89
## 23	Mississippi	53
## 24	Missouri	66
## 25	Montana	15
## 26	Nebraska	38
## 27	Nevada	39
## 28	New Hampshire	27
## 29	New Jersey	130
## 30	New Mexico	37
## 31	New York	1128
## 32	North Carolina	249
## 33	North Dakota	7
## 34	Ohio	469
## 35	Oklahoma	66
## 36	Oregon	124
## 37	Pennsylvania	587
## 38	Rhode Island	56
## 39	South Carolina	42
## 40	South Dakota	12
## 41	Tennessee	183
## 42	Texas	985
## 43	Utah	53
## 44	Vermont	11
## 45	Virginia	224
## 46	Washington	506
## 47	West Virginia	4
## 48	Wisconsin	110
## 49	Wyoming	1

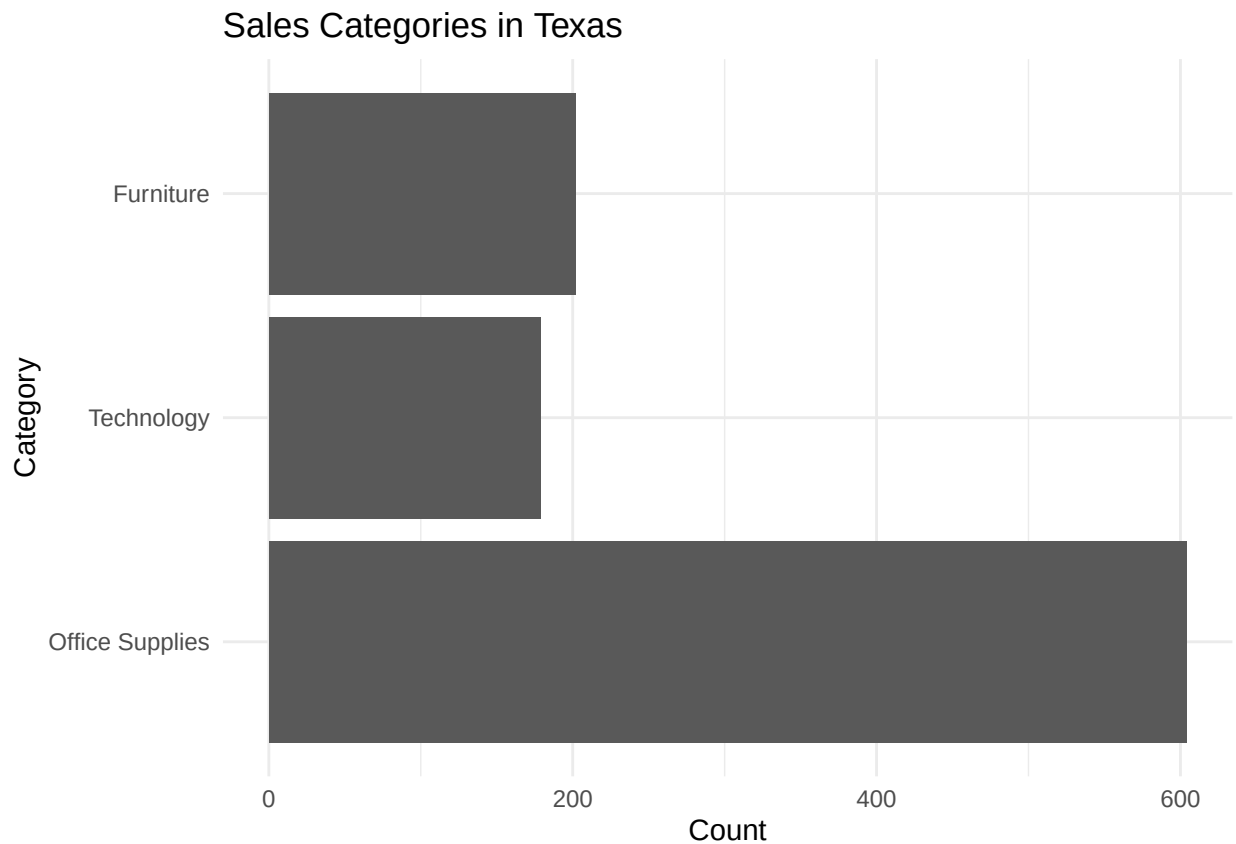
Exercise 8

Create an alphabetically ordered barplot for each sales Category in the State of Texas.

```
library(ggplot2)

# Filter State of Texas
texas_data <- subset(data, State == "Texas")
```

```
# Create the barplot
ggplot(texas_data, aes(x = fct_inorder(Category))) +
  geom_bar() +
  labs(title = "Sales Categories in Texas",
       x = "Category",
       y = "Count") +
  theme_minimal() +
  coord_flip()
```



Exercise 9

Find the average profit by region. **Note: You will need to use the `aggregate()` function to do this. To understand how the function works type `?aggregate` in the console.**

```
# Calculate the average profit by region
average_profit_by_region <- aggregate(Profit ~ Region, data = data, FUN = mean, na.rm = TRUE)

# Print
print(average_profit_by_region)
```

```
##      Region  Profit
## 1 Central 17.21514
## 2   East 32.23360
```

```
## 3    South 29.00771
## 4     West 33.98985
```

Exercise 10

Find the average profit by order year. **Note:** You will need to use the `aggregate()` function to do this. To understand how the function works type `?aggregate` in the console.

```
# Extract the order year from Order.Date
data$Order.Year <- year(data$Order.Date)

# Average
average_profit_by_year <- aggregate(Profit ~ Order.Year, data = data, FUN = mean, na.rm = TRUE)

# Print
print(average_profit_by_year)
```

```
##   Order.Year   Profit
## 1      2014 24.93035
## 2      2015 29.44158
## 3      2016 31.79268
## 4      2017 28.32405
```